

LAPORAN UJIAN AKHIR SEMESTER
Object-Oriented Programming
Aplikasi Sistem Informasi Form Pemesanan Berbasis Java Swing dan
Database MySQL
Dosen Pengampu : I Made Gede Sri Artha, S.T.



Disusun Oleh :

Nama : I Nyoman Widana
NIM : 2401010021
Jurusan : Manajemen Data & Informasi
Kelas : A

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT BISNIS DAN TEKNOLOGI INDONESIA
TAHUN PELAJARAN 2025/2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data transaksi pada sebuah unit usaha atau sistem pemesanan barang memerlukan tingkat akurasi dan efisiensi yang tinggi. Proses pencatatan yang bersifat konvensional rawan terhadap kesalahan manusia (*human error*), seperti kekeliruan pencatatan tanggal, hilangnya riwayat pesanan.

Untuk mengatasi permasalahan tersebut, dikembangkan sebuah aplikasi desktop bernama **Sistem Form Pemesanan**. Aplikasi ini dibangun dengan memanfaatkan bahasa pemrograman Java melalui pendekatan Pemrograman Berorientasi Objek (PBO) secara penuh. Antarmuka aplikasi memanfaatkan framework grafis javax.swing (Java Swing JFrame) untuk memberikan kemudahan interaksi bagi pengguna (*user-friendly*). Seluruh data transaksi diintegrasikan secara terpusat dan disimpan ke dalam sistem manajemen basis data relasional MySQL melalui konektivitas JDBC (*Java Database Connectivity*).

1.2 Tujuan

- Merancang dan mengimplementasikan sistem informasi pengelolaan data pemesanan yang mendukung fungsionalitas CRUD (*Create, Read, Update, Delete*) secara *real-time*.
- Menerapkan pilar-pilar utama Pemrograman Berorientasi Objek, khususnya konsep **Encapsulation** (Enkapsulasi) dan **Polymorphism** melalui mekanisme *Constructor Overloading*.
- Membangun mekanisme kontrol antarmuka yang cerdas berupa komponen *Dynamic Date ComboBox* untuk validasi otomatis jumlah hari berdasarkan kombinasi bulan dan tahun pilihan pengguna.

BAB II LANDASAN TEORI & PERANCANGAN SISTEM

2.1 Pemrograman Berorientasi Objek (PBO)

Pemrograman Berorientasi Objek (PBO) atau *Object-Oriented Programming* (OOP) adalah pilar metodologi pemrograman yang menyusun arsitektur perangkat lunak berdasarkan interaksi komponen objek digital di dalam memori.

1. **Encapsulation (Enkapsulasi):** Adalah mekanisme pembungkusan data dan penyembunyian variabel status internal (*state*) dari interaksi langsung pihak luar kelas. Variabel dideklarasikan secara tertutup (*private*), dan modifikasi nilai hanya diizinkan melalui saluran perantara yang valid, yaitu metode fungsi akses *Getter* dan *Setter*.
2. **Polymorphism (Polimorfisme):** Karakteristik objek yang memungkinkan satu nama entitas kode memiliki banyak bentuk perilaku yang berbeda. Dalam tingkat kompilasi (*compile-time polymorphism*), konsep ini diwujudkan lewat teknik **Method/Constructor Overloading**, di mana sebuah kelas memiliki lebih dari satu konstruktor/metode dengan nama yang sama, tetapi dibedakan atas jumlah atau tipe parameter yang diterima.

2.2 Desain Basis Data (MySQL)

Sistem menggunakan database relasional bernama pemesanan dengan tabel master bernama pesanan. Spesifikasi struktur skema tabel dirancang sebagai berikut:

Nama Kolom	Tipe Data	Atribut	Keterangan
id	INT	Primary Key, Auto Increment	ID Unik Transaksi Pesanan
produk	VARCHAR(128)	Not Null	Nama komponen produk
jumlah	INT	Not Null	Kuantitas jumlah barang
harga	DECIMAL (16, 2)	Not Null	Nilai nominal harga per produk

tipe	ENUM (‘online’, ‘offline’)	Not Null	Metode pemesanan (Online / Offline)
tanggal	DATE	Not Null	Waktu pencatatan nota masuk
deadline	DATE	Not Null	Target batas waktu penyelesaian
kategori	ENUM (‘elektronik’, ‘fashion’, ‘makanan’, ‘peralatan’)	Not Null	Kelompok kategori produk

BAB III IMPLEMENTASI & PEMBAHASAN KODE

3.1 Implementasi Encapsulation (Enkapsulasi)

Enkapsulasi diterapkan secara menyeluruh di dalam kelas model entitas data Pesanan.java. Seluruh variabel anggota yang merepresentasikan data pesanan dikunci menggunakan modifier akses private.

```
package formpemesanan;

public class Pesanan {
    // Atribut dideklarasikan secara private untuk perlindungan data
    private int id;
    private String NamaProduk;
    private int JumlahProduk;
    private double HargaProduk;
    private String TipePemesanan;
    private String TanggalPemesanan;
    private String TanggalDeadline;
    private String KategoriProduk;

    // Manajemen pembacaan data dilayani secara terstruktur via fungsi public Getter
    public int getId() {
        return id;
    }
    public String getNamaProduk() {
        return NamaProduk;
    }
    public int getJumlahProduk() {
        return JumlahProduk;
    }
    public double getHargaProduk() {
        return HargaProduk;
    }
    public String getTipePemesanan() {
        return TipePemesanan;
    }
    public String getTanggalPemesanan() {
        return TanggalPemesanan;
    }
    public String getTanggalDeadline() {
        return TanggalDeadline;
    }
}
```

```

    public String getKategoriProduk() {
        return KategoriProduk;
    }
}

```

3.2 Implementasi Polymorphism (Constructor Overloading)

Proyek ini mendemonstrasikan aspek polimorfisme melalui penciptaan konstruktor ganda (*Constructor Overloading*) di dalam kelas Pesanan.java.

```

// Konstruktor Bentuk 1: Tanpa Parameter ID (Digunakan untuk Entri Data Baru dari
Form)
public Pesanan(String NamaProduk, int JumlahProduk, double HargaProduk, String
TipePemesanan, String TanggalPemesanan, String TanggalDeadline, String
KategoriProduk) {
    this.NamaProduk = NamaProduk;
    this.JumlahProduk = JumlahProduk;
    this.HargaProduk = HargaProduk;
    this.TipePemesanan = TipePemesanan;
    this.TanggalPemesanan = TanggalPemesanan;
    this.TanggalDeadline = TanggalDeadline;
    this.KategoriProduk = KategoriProduk;
}

// Konstruktor Bentuk 2: Menggunakan Parameter ID (Overloading - Digunakan saat
Mapping dari ResultSet Database)
public Pesanan(int id, String NamaProduk, int JumlahProduk, double HargaProduk,
String TipePemesanan, String TanggalPemesanan, String TanggalDeadline, String
KategoriProduk) {
    this.id = id;
    this.NamaProduk = NamaProduk;
    this.JumlahProduk = JumlahProduk;
    this.HargaProduk = HargaProduk;
    his.TipePemesanan = TipePemesanan;
    this.TanggalPemesanan = TanggalPemesanan;
    this.TanggalDeadline = TanggalDeadline;
    this.KategoriProduk = KategoriProduk;
}

```

Analisis: **Konstruktor Bentuk 1** dipicu ketika user menekan tombol simpan baru; sistem mengabaikan ID karena MySQL akan mengisinya lewat fitur *Auto-Increment*. Sebaliknya, **Konstruktor Bentuk 2** dipanggil ketika perulangan `while(rs.next())` membaca database

untuk memetakan record lama ke dalam memori aplikasi sebelum dilempar ke komponen visual JTable.

3.3 Pembahasan Mekanisme CRUD Sistem

Operasi data utama dikelola melalui interaksi langsung antara kontrol komponen GUI `FrameFormPemesanan` dan kueri SQL terenkapsulasi yang tertulis pada `Pesanan.java`.

3.3.1 Create (Operasi Penambahan Data)

Ketika user mengisi formulir dan menekan tombol Simpan dalam kondisi data baru, aplikasi menangkap input tersebut lalu instansiasi objek `Pesanan` menggunakan **Konstruktor Bentuk 1**. Objek tersebut kemudian memanggil metode instans `.save()`.

```
// Metode untuk menyimpan pesanan baru ke database
public void save() {
    try (Connection conn = connect();
        PreparedStatement stmt = conn.prepareStatement("INSERT INTO pesanan (produk,
jumlah, harga, tipe, tanggal, deadline, kategori) VALUES (?, ?, ?, ?, ?, ?, ?)")) {
        stmt.setString(1, NamaProduk);
        stmt.setInt(2, JumlahProduk);
        stmt.setDouble(3, HargaProduk);
        stmt.setString(4, TipePemesanan);
        stmt.setString(5, TanggalPemesanan);
        stmt.setString(6, TanggalDeadline);
        stmt.setString(7, KategoriProduk);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

3.3.2 Read (Operasi Pembacaan & Penayangan Data)

Mekanisme penayangan data dipicu oleh metode refreshPesanan() yang mengeksekusi fungsi statik Pesanan.all(). Fungsi ini menarik seluruh data dari database dan memetakan baris ResultSet ke dalam array objek menggunakan **Konstruktur Bentuk 2** (termasuk ID), kemudian didistribusikan ke komponen JTable.

```
public static Pesanan[] all() {
    Pesanan[] pesananList = null;
    int index = 0;
    try (Connection conn = connect());
        PreparedStatement stmt = conn.prepareStatement("SELECT * FROM pesanan
ORDER BY tanggal ASC");
        ResultSet rs = stmt.executeQuery() {
            while (rs.next()) {
                int id = rs.getInt("id");
                String namaProduk = rs.getString("produk");
                int jumlahProduk = rs.getInt("jumlah");
                double hargaProduk = rs.getDouble("harga");
                String tipePemesanan = rs.getString("tipe");
                String tanggalPemesanan = rs.getString("tanggal");
                String tanggalDeadline = rs.getString("deadline");
                String kategoriProduk = rs.getString("kategori");
                Pesanan pesanan = new Pesanan(id, namaProduk, jumlahProduk,
hargaProduk, tipePemesanan, tanggalPemesanan, tanggalDeadline, kategoriProduk);
                if (pesananList == null) {
                    pesananList = new Pesanan[1];
                } else {
                    Pesanan[] tempList = new Pesanan[pesananList.length + 1];
                    System.arraycopy(pesananList, 0, tempList, 0,
pesananList.length);
                    pesananList = tempList;
                }
                pesananList[index++] = pesanan;
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return pesananList;
    }
```


3.3.3 Update (Operasi Pengubahan Data)

Jika salah satu baris pada tabel diklik, variabel `selectedPesananId` akan menangkap nilai ID baris bersangkutan. Saat tombol Simpan ditekan kembali, program mendeteksi bahwa operasi yang berjalan adalah proses penyuntingan, lalu mengeksekusi metode `.update(id)`.

```
public void update(int id) {
    try (Connection conn = connect();
        PreparedStatement stmt = conn.prepareStatement("UPDATE pesanan SET produk
= ?, jumlah = ?, harga = ?, tipe = ?, tanggal = ?, deadline = ?, kategori = ?
WHERE id = ?")) {
        stmt.setString(1, NamaProduk);
        stmt.setInt(2, JumlahProduk);
        stmt.setDouble(3, HargaProduk);
        stmt.setString(4, TipePemesanan);
        stmt.setString(5, TanggalPemesanan);
        stmt.setString(6, TanggalDeadline);
        stmt.setString(7, KategoriProduk);
        stmt.setInt(8, id);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

3.3.4 Delete (Operasi Penghapusan Data)

Operasi penghapusan dilakukan secara statik dengan melemparkan parameter ID target setelah mendapatkan konfirmasi dialog visual dari pengguna (`JOptionPane`).

```
public static void delete(int id) {
    try (Connection conn = connect();
        PreparedStatement stmt = conn.prepareStatement("DELETE FROM pesanan
WHERE id = ?")) {
        stmt.setInt(1, id);
        stmt.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

3.4 Logika Algoritma Dynamic Date ComboBox

Guna mengeliminasi kesalahan penginputan tanggal yang tidak valid (contoh: 31 November atau 29 Februari di tahun non-kabisat), dibuatlah metode cerdas bernama `sesuaikanTanggal()` di dalam file `FrameFormPemesanan.java`. Fungsi ini memanfaatkan class API modern `java.time.YearMonth`.

```
private void sesuaikanTanggal(javax.swing.JComboBox<String> cbxThn,
    javax.swing.JComboBox<String> cbxBln, javax.swing.JComboBox<String> cbxTgl) {
    // Membaca komposisi kalender dinamis berdasarkan bulan dan tahun pilihan
    YearMonth yearMonth = YearMonth.of(tahunTerpilih, bulanTerpilih);
    int maxHari = yearMonth.lengthOfMonth(); // Mendapatkan total hari riil (28, 29,
    30, atau 31)

    cbxTgl.removeAllItems();
    for (int i = 1; i <= maxHari; i++) {
        cbxTgl.addItem(String.valueOf(i));
    }
}
```

Metode ini diikat ke dalam event `ItemStateChanged` pada ComboBox Bulan dan Tahun, sehingga setiap kali user mengganti bulan atau tahun, kapasitas isi ComboBox Tanggal langsung menyesuaikan secara otomatis.

3.5 Mekanisme Reset Form & Refresh Sistem

Pembersihan formulir masukan dan pemuatan ulang tabel diatur di dalam event tombol Refresh (`btnRefreshMouseClicked`). Alurnya berjalan sebagai berikut:

1. Memanggil fungsi `clearForm()` untuk mengembalikan string kosong "" pada `tfNamaProduk` dan `tfHargaPerProduk`.
2. Mengembalikan indeks pilihan seluruh ComboBox (`cbxTanggal`, `cbxBulan`, `cbxKategori`) kembali ke posisi indeks default 0.
3. Menyetel ulang variabel penunjuk baris aktif `selectedPesananId = -1` (mengindikasikan form tidak dalam keadaan menyeleksi baris tabel manapun).

4. Mengeksekusi fungsi refreshPesanan() untuk memicu ulang pembacaan kueri SELECT ke database, mendistribusikan data terbaru ke DefaultTableModel, dan memperbarui UI komponen JTable.

BAB IV PENUTUP

4.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa aplikasi **Form Pemesanan** ini berhasil dibangun dengan memenuhi standar metodologi Pemrograman Berorientasi Objek secara utuh. Penerapan sifat *Encapsulation* terbukti mampu mengamankan integritas variabel model objek data dari modifikasi eksternal yang tidak sah. Penggunaan teknik *Constructor Overloading* (Polimorfisme) memberikan fleksibilitas tinggi bagi program untuk membedakan perlakuan instansiasi data baru dan pemetaan data lama dari database MySQL.

Aplikasi juga berjalan dengan tingkat stabilitas yang baik dalam memproses siklus operasi CRUD, didukung sinkronisasi form terintegrasi lewat tombol Refresh serta validasi input cerdas melalui fitur *Dynamic Date ComboBox*.